

Vendas Oficiais por Mês e País

Case de Análise de Dados

Leonardo Ruiz

Código e documentação: <https://github.com/leonardocarr/stays-vendas-oficiais>

O que este documento traz	
Parte 1	Modelo em camadas, o SQL da camada de consumo e três riscos de qualidade
Parte 2	Uso de IA no processo, com prompts e critério, e proposta de documentação
Parte 3	Nota de comunicação para liderança não técnica
Anexo	Achados adicionais na base e instruções para rodar

Os números citados ao longo do documento foram apurados sobre a base recebida e podem ser reproduzidos rodando o projeto. O extrato entregue tem 43 linhas; a lógica não assume volume e roda igual sobre as cerca de 2.000 linhas descritas no enunciado.

O problema

O CRM registra tudo como oportunidade: a venda nova, o upgrade de plano de quem já é cliente e a reativação de quem tinha cancelado. Os três trazem receita e o relatório financeiro soma os três. Já a pergunta “o comercial está trazendo cliente novo, e em que ritmo?” precisa de um número diferente, e somar upgrade junto responde errado essa pergunta.

O objetivo do pipeline é produzir esse segundo número de forma confiável e auditável, a partir de um extrato bruto com país escrito de várias formas, datas em formatos distintos, duplicidades e registros que não são venda comercial misturados.

Parte 1 — Estruturação

1. Modelo em camadas

Três camadas, no padrão bruto, tratado e consumo, que a literatura também chama de bronze, silver e gold.

Camada	Grão	Chave	O que trata
Bruto (bronze)	1 linha = 1 registro como veio do CRM	Não tem. O arquivo traz id repetido e id vazio.	Nada. O CSV é lido com todas as colunas como texto, sem conversão. Serve de fonte de auditoria: quando um número parecer estranho adiante, é aqui que se confere o que o CRM mandou de fato.
Tratado (silver)	1 linha = 1 oportunidade, já sem duplicatas	Composta: id_oportunidade + conta + data_venda. As três juntas, porque o id sozinho não dá conta.	Normaliza país e estágio, converte data e valor, remove duplicatas e classifica em nova_venda, upgrade ou reativacao. Marca as linhas problemáticas com data_ambigua e mrr_ausente. Mantém o grão de oportunidade, então dá para abrir qualquer número da camada seguinte e ver o que o compõe. O que é descartado vai para um log à parte, com o motivo.
Consumo (gold)	1 linha = ano_mes × país × tipo_venda	ano_mes + país + tipo_venda	Filtra só o que está como Ganho, tira os upgrades da contagem e agrega quantidade e MRR. É a camada que vai para o BI. Quem consome não precisa saber que existe um país escrito como “Hispan” na origem.

Uma regra atravessa as três camadas: **a venda conta pela data de fechamento**, não pela data de criação da oportunidade. É o que faz um negócio fechado em fevereiro aparecer em fevereiro, mesmo tendo entrado no funil em janeiro.

Sobre a escolha

A camada tratada já está no grão de uma tabela fato, com país, data, classe e MRR como atributos. A camada de consumo é uma agregação possível sobre ela, não a única. Se amanhã pedirem a quebra por plano ou por SDR, não é preciso mexer no pipeline: escreve-se outra agregação sobre a mesma base.

Considere e descartei duas alternativas. **Star schema** com dimensões separadas de país, calendário e conta não se paga aqui: com três países e uma fonte pequena, essas dimensões seriam listas de dois ou três valores sem atributo próprio, adicionando join sem devolver nada. **dbt** é o mesmo modelo em camadas como framework, com teste e documentação automáticos; para um CSV rodado uma vez custa mais do que entrega, mas seria a primeira migração se o pipeline virasse recorrente.

2. SQL da camada de consumo

A consulta completa está em `sql/gold_vendas_oficiais.sql` no repositório, escrita em CTEs encadeadas que vão do CSV bruto até a agregação. Abaixo, onde cada uma das quatro regras é aplicada.

Regra pedida	Onde é aplicada	Conferência na saída
Normalizar país	Mapa explícito em <code>silver_normalizado</code> . O ELSE cai em NULL para que grafia nova apareça como não mapeada em vez de ser classificada errado.	8 grafias na origem viram 3 países. Nenhuma linha chega à saída com país nulo.
Excluir upgrades da contagem	Classificação em <code>silver_classificado</code> , pelo preenchimento de <code>plano_anterior_upgrade</code> .	upgrade não aparece na saída. Conferi que as outras três colunas de upgrade só aparecem junto com essa, então uma basta como flag.
Separar reativações sem sumir com elas	Mesma classificação, por <code>sdr_perfil</code> igual a Customer Success - Retenção. Saem em linha própria via <code>tipo_venda</code> .	As 4 reativações da base aparecem na saída, em linhas separadas. Nenhuma foi parar no log de descartes.
Definir a venda pela data de fechamento	<code>data_da_venda</code> é a única origem de <code>data_venda</code> . O filtro final exige data válida.	Não há outro campo de data na base, então não há risco de confundir com data de criação.

A agregação final, que produz a camada de consumo:

```
SELECT
    strftime(data_venda, '%Y-%m') AS ano_mes,
    pais,
    classe                         AS tipo_venda,
    COUNT(*)                      AS qtd_vendas,
    SUM(mrr)                      AS mrr_total
FROM silver
WHERE classe IN ('nova_venda', 'reativacao')
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

A classificação que sustenta as regras 2 e 3:

```
CASE
    WHEN COALESCE(plano_anterior_upgrade, '') <> '' THEN 'upgrade'
    WHEN sdr_perfil = 'Customer Success - Retenção' THEN 'reativacao'
    ELSE 'nova_venda'
END AS classe
```

O COALESCE existe porque a coluna vazia chega do CSV como NULL. Sem ele, a condição funcionaria por efeito colateral de NULL se comportar como falso dentro do CASE, e não porque o código diz o que quer. Continuaría correta até alguém reescrever a condição na forma negativa, quando passaria a errar em silêncio.

Resultado

A saída tem 16 linhas, somando 19 vendas novas e 4 reativações, sem nenhum upgrade. A mesma lógica foi escrita duas vezes, em SQL puro e em pandas (notebooks/pipeline.py), e as duas produzem resultado idêntico. Usei isso como conferência antes de reportar qualquer número: se divergissem, haveria bug em uma delas.

ano_mes	pais	tipo_venda	qtd_vendas	mrr_total
2026-01	Brasil	nova_venda	2	1380.0
2026-02	Brasil	nova_venda	5	3470.0
2026-03	Brasil	reativacao	1	390.0
2026-03	Espanha	nova_venda	1	1400.0
...				(16 linhas no total)

3. Três riscos de qualidade no dado bruto

1. A chave de negócio não é confiável

Em 43 linhas, uma tem id_oportunidade vazio e dois ids aparecem repetidos. Sem chave única garantida, dá para contar a mesma venda duas vezes ou jogar fora uma venda real.

Tratamento. Deduplicação pela chave composta de id, conta e data, mantendo entre as duplicatas o registro mais completo. O total removido é impresso na execução e as linhas descartadas vão para um log. Esse critério de desempate não é detalhe: no id 1008 as duas cópias eram iguais exceto pelo MRR, uma com 990,00 e a outra vazia. Sem ele, R\$ 990 saíam do relatório em silêncio.

2. As datas vêm em formatos misturados, e parte delas é ambígua

O campo traz “14-fev.-2026”, o formato ISO e o formato com barras. Nas barras não há indicação de ser DD/MM ou MM/DD, e quando dia e mês são ambos menores que 13 as duas leituras são válidas e caem em meses diferentes. O efeito é venda no mês errado, distorcendo tendência mensal e comissionamento.

Tratamento. Adoto DD/MM como padrão e inverte apenas quando o segundo número passa de 12, o que torna DD/MM impossível. As linhas realmente ambíguas recebem a flag data_ambigua em vez de serem resolvidas em silêncio: são 10 das 43, e nelas as duas leituras sempre caem em meses diferentes, afetando R\$ 5.380 em vendas ganhas. A saída definitiva é conferir essas datas na origem; a flag existe para que alguém saiba que precisa conferir.

3. As categorias foram digitadas de formas diferentes

São 8 grafias distintas em país para 3 países, incluindo BRASIL, Brasil, brasil, BR, ESPANHA, Espanha e Hispan. Em estágio são 7 valores, misturando português e inglês. Isso sub ou superestima vendas por país e pode fazer uma venda ganha não ser reconhecida como tal.

Tratamento. Um mapa de normalização único, em um só lugar do código, com saída explícita para NULL quando o valor não é reconhecido. Assim uma grafia nova aparece como não mapeada, em vez de ser descartada ou classificada errado por acidente.

Premissas em aberto

Três decisões foram tomadas por falta de quem confirmasse, e estão marcadas como suposição no código, em um único ponto cada: “Hispan” tratado como Espanha; data ambígua lida como DD/MM; e “R\$ 1.400” valendo 1400 e não 1,4.

Parte 2 — IA na rotina

4. Em que etapas usei, com que abordagem

Levantamento inicial do dado

Antes de escrever qualquer regra, pedi os valores distintos de cada coluna categórica e o agrupamento das variações que pareciam significar a mesma coisa. É tarefa mecânica e o resultado é conferível abrindo o CSV.

Liste os valores distintos de país, estagio, sdr_perfil e plano_anterior_upgrade neste CSV. Agrupe as variações que provavelmente são a mesma coisa (maiúscula, abreviação, idioma diferente). Não decida o mapeamento final, só aponte os candidatos.

O detalhe que importa é o final: candidatos, não decisão. “Hispan” provavelmente é Espanha, mas segue sendo palpite até alguém do negócio confirmar.

Primeira versão do código

A divisão aqui é a que uso sempre: eu defino a regra, a ferramenta escreve a sintaxe. Não peço “trate as datas desse arquivo”, porque isso entrega a decisão. Peço a implementação de uma regra que já decidi.

Escreva uma função Python que converta para data estes três formatos: “14-fev.-2026”, “2026-05-08” e “01/02/2026”. No terceiro, assuma DD/MM e inverta para MM/DD apenas quando o segundo número passar de 12. Retorne nulo quando não reconhecer o formato, sem levantar exceção.

O ganho é tempo de regex e de função de data. O que não delego é a regra: quem decidiu que o padrão é DD/MM fui eu. Todo rascunho passou por teste contra o dado real antes de entrar, e foi isso que revelou dois erros que ninguém pega lendo: a heurística de data comparava o número errado e quebrava em “04/15/2026”, e o parser de moeda lia “R\$ 1.400” como 1,4. Nenhum dos dois levantava exceção.

Documentação

Depois que o schema estabilizou, usei para transformar schema e comentários em texto legível. O prompt carrega uma trava:

A partir deste schema e dos comentários do código, escreva uma tabela com uma linha por coluna: nome, tipo e descrição. Use apenas o que está no código. Onde a intenção não estiver clara, escreva “não documentado” em vez de deduzir.

Sem a última frase, o que volta é plausível e inventado. Uma coluna chamada valor_nmrr_upgrade ganha uma descrição convincente sobre receita incremental que ninguém confirmou. “Não documentado” é uma resposta útil; um palpite bem escrito, não.

Onde não confiei sem conferir

- **Regra de negócio com efeito em dinheiro.** Se “Hispan” é Espanha, se reativação entra como venda oficial, se a data de fechamento é a referência: é decisão de quem responde pelo número. Por isso as premissas estão marcadas como suposição, em vez de terem virado fato só por funcionarem no teste.
- **Qualquer coisa que apaga linha.** Uma regra de dedup plausível pode remover uma venda legítima. Conferi quantas linhas entraram, quantas saíram e quais especificamente foram descartadas. Foi assim que o caso do id 1008 apareceu.
- **Número que vai para outra pessoa.** Escrevi a agregação duas vezes, em pandas e em SQL, e só usei o resultado depois que as duas bateram.
- **Classificação que afeta meta ou comissão.** O que conta como venda oficial mexe com meta de time e remuneração de SDR, e precisa de aval de quem é dono da métrica.

O critério, na prática

Antes de aceitar qualquer saída, faço uma pergunta só: **se isso estiver errado, como eu descubro?**

Se a resposta for “rodando contra o dado” ou “relendo o código”, uso sem medo, porque tenho como verificar. Se a resposta for “só saberia se alguém do negócio me contasse”, não uso para decidir; uso no máximo para listar as opções, e a escolha fica registrada como pendência. É a diferença entre a ferramenta acelerar o trabalho e ela tomar decisão no lugar de quem responde por ela.

5. Bônus: documentar o modelo sem virar gargalo

O problema que essa proposta resolve não é escrever o dicionário de dados. É ele continuar verdadeiro depois. Documentação escrita à parte envelhece: alguém muda a regra, esquece o documento, e em três meses ele mente. Quando isso acontece, todos voltam a perguntar para quem escreveu o pipeline, que é exatamente a dependência que se queria eliminar.

A proposta é gerar o dicionário a partir do código, com quatro partes:

- **A fonte da verdade é o código, não um documento paralelo.** O que alimenta a geração é o schema de cada camada mais os comentários que registram decisão, incluindo os marcadores de suposição. Nada de manter descrição de coluna em planilha separada.
- **A geração entra no mesmo commit da mudança.** Quando uma alteração mexe em schema ou regra, o dicionário é regerado e o diff vai junto. A documentação nunca fica mais de um commit atrás do código, e não depende de alguém lembrar.
- **O prompt proíbe inferência.** Descreva o que está no código e escreva “não documentado” no resto. O maior risco não é errar o tipo de uma coluna, é inventar significado de negócio com texto convincente.
- **Uma pessoa revisa antes de entrar.** A ferramenta faz o rascunho, alguém confirma. Leva minutos e é o que separa documentação gerada de documentação confiável.

O que decide se alguém consome sem me procurar não é a lista de colunas. É o dicionário responder as quatro perguntas que fazem a pessoa vir perguntar:

- O que cada coluna significa e de que tipo é.
- Qual o grão e qual a chave, para ninguém fazer join errado e duplicar linha sem perceber.
- Que pergunta a tabela responde e, principalmente, **qual ela não responde**. É o que evita alguém usar `mrr_total` achando que é a receita do mês.
- O que foi descartado, por quê, e quais premissas ainda não foram confirmadas.

As duas últimas são as que costumam gerar a pergunta. “Por que esse número não bate com o financeiro?” e “cadê a venda da conta X?” têm resposta escrita no dicionário, no log de descartes e na nota de comunicação, sem depender de mim.

Em escala, essa rotina manual é substituída pelo que o dbt já faz nativamente: documentação e lineage gerados a partir dos próprios modelos. A versão descrita aqui é o mesmo princípio no tamanho deste projeto.

Parte 3 — Comunicação

6. Por que “vendas oficiais” não bate com o relatório financeiro

Texto destinado a liderança não técnica.

Os dois relatórios respondem perguntas diferentes. É por isso que mostram números diferentes, e nenhum dos dois está errado.

O **financeiro** responde quanto dinheiro entrou no mês. Para essa pergunta faz sentido somar tudo: o cliente novo que assinou o plano Start, o cliente antigo que subiu de Start para Business e o que tinha cancelado e voltou. Os três trazem receita, então os três entram.

O de **vendas oficiais** responde outra coisa: se o time comercial está trazendo cliente novo, e em que ritmo. Aí somar upgrade atrapalha, porque upgrade é cliente que já estava na casa e resolveu gastar mais. Um mês com muitos upgrades e pouca venda nova pareceria um bom mês de vendas, quando quem entregou foi o time que cuida da carteira existente. São times e metas diferentes.

Pensa numa academia contando alunos novos. Quem já treina lá e migra para o plano premium não é aluno novo, mesmo pagando mais. Se contar essa migração como aluno novo, a academia vai achar que está captando mais do que está, e pode contratar mais gente para a recepção esperando um fluxo que não existe.

A reativação segue a mesma lógica. Cliente que cancelou e voltou é resultado bom, mas de outra frente: recuperar quem saiu, não conquistar quem nunca veio. Por isso ela aparece em linha própria, sem ser somada às vendas novas e sem ser descartada.

Na prática, se o financeiro fecha o mês com R\$ 50 mil e as vendas oficiais mostram R\$ 38 mil, a maior parte da diferença é upgrade e reativação.

Uma ressalva: nem toda a diferença é intencional. Algumas vendas já fechadas ficam de fora porque a data de fechamento não foi preenchida no sistema. É dinheiro real saindo da conta por falha de cadastro, não por regra. Essas vendas estão identificadas e separadas, e resolver depende de preencher o campo na origem.

Anexo — Achados adicionais

Estes não foram pedidos. Apareceram durante a análise e afetam a leitura do número, por isso ficam registrados.

Vendas ganhas que não entram no relatório

Das 43 linhas, 15 não chegam à camada de consumo. Nove são exclusões de negócio esperadas (oportunidade perdida ou ainda em aberto) e duas são duplicatas. As outras **cinco são vendas marcadas como Ganho no CRM, mas sem data de fechamento preenchida**. Como a regra define a venda pela data de fechamento, elas ficam de fora: são R\$ 5.860 de MRR já fechado que não aparecem no relatório, não porque alguém decidiu excluir, mas porque um campo não foi preenchido.

Esse caso só ficou visível porque o log de descartes separa por motivo. Uma contagem agregada de “15 linhas removidas” teria escondido a diferença entre uma exclusão correta e receita sumindo por falha de cadastro.

A identificação de reativação depende de um campo que vem vazio

A regra só reconhece reativação quando `sdr_perfil` é “Customer Success - Retenção”. Esse campo vem vazio em 9 das 43 linhas, e sem ele a linha cai em `nova_venda` por descarte, não por evidência. Hoje **6 das 19 vendas novas do relatório** estão nessa situação: se alguma for reativação, está inflando aquisição e escondendo recuperação de clientes.

Não dá para resolver no código, porque o dado que decidiria não está no arquivo. O tratamento honesto é manter a classificação, deixar registrado que ela vale enquanto o campo estiver preenchido, e pedir ao time do CRM ou a obrigatoriedade do campo, ou um flag próprio de reativação, como o upgrade já tem. Identificar o que foi a venda por quem atendeu é frágil: basta uma reativação ser tocada por outro time para virar venda nova sem ninguém perceber.

Outros pontos

- **Seis vendas entram no relatório sem valor de MRR.** A contagem delas está certa, mas a soma ignora nulo em silêncio, então o total do grupo fica menor que a realidade, ou nulo quando nenhuma linha do grupo tem valor. A flag `mrr_ausente` marca essas linhas.
- **Oportunidades ainda em aberto aparecem com data preenchida**, uma delas no futuro. Sugere que o campo mistura data de fechamento real com data prevista. Não afeta o número atual, porque o filtro de Ganho barra essas linhas antes da agregação, e confirmei que nenhuma venda ganha tem data futura. Mas é ambiguidade de campo que vale confirmar na origem.

Como rodar

O projeto não exige banco de dados. O DuckDB lê o CSV direto.

```
pip install -r requirements.txt

python notebooks/pipeline.py      # pipeline completo, em pandas
python sql/run_sql.py            # só a camada de consumo, via SQL
python sql/run_rejeitados.py     # log do que foi descartado, com motivo
```


Organização do repositório

Caminho	Conteúdo
sql/gold_vendas_oficiais.sql	A consulta completa, do CSV bruto até a agregação
notebooks/pipeline.py	A mesma lógica em pandas, usada para conferência cruzada
sql/explorar_sql.py	Inspeção do SQL por etapas, uma célula por CTE
docs/data_dictionary.md	Dicionário das três camadas, coluna por coluna
docs/metodologia.md	Por que camadas, e alternativas descartadas
docs/uso_de_ia.md	Etapas, prompts e critério
docs/comunicacao_lideranca.md	A nota da Parte 3

Repositório: <https://github.com/leonardocarr/stays-vendas-oficiais>